# CSCE 355: Turing Machine Assignment 1

## Overview

In this assignment you will build six deterministic single-tape Turing machines in JFLAP: four **decision** machines (Problems 1–4) that accept or reject an input string, and two **transducer** machines (Problems 5–6) that compute and leave a specific output on the tape. Together they cover the basic building blocks you will reuse throughout the course: scanning left to right, finding the end of the input, remembering information in your states, and writing symbols to the tape (for Turing machines, stack for PDAs).

You will submit six `.jff` files. Each will be tested automatically against provided test data, worth the point value listed with each problem below, for 25 points total.

### Getting JFLAP

This assignment uses JFLAP, a free tool for building and simulating automata. Download it directly from the official site:

`https://www.jflap.org`

JFLAP is a `.jar` file and requires a working Java installation (Java 8 or later is recommended). It is not included in this assignment package – download it yourself from the link above. Do not use a third-party mirror. To start it, assuming you're in the directory, type

**java -jar JFLAP7.1.jar**

### Getting Python 3

To run the test python file will need Python 3 installed. Search for how to do this for your operating system (tested on Linux).

## Determinism Requirement

**Every machine you submit must be deterministic.** If a machine is found to be nondeterministic, it receives **zero credit** for that problem, regardless of whether it produces correct results on the test data. This is checked automatically by the grading script before any test data is even run.

A Turing machine is deterministic if, for every state and every tape symbol, there is *at most one* transition that applies. Concretely, watch out for this situation: **two transitions leaving the same state on the same read symbol.** If state $q_i$ has two different outgoing transitions that both read the symbol `0` (even if they write different symbols, move in different directions, or go to different states), the machine is nondeterministic at $q_i$. In a standard JFLAP Turing machine every transition reads some tape symbol (including the blank symbol, shown as $\square$), so this is really

the only case to watch for – but check every state with more than one outgoing arrow to make sure no two of them share a read symbol.

### How to check for determinism in JFLAP

**Method 1: Test → Highlight Nondeterminism.** With your machine open in the editor, use the menu option **Test → Highlight Nondeterminism**. JFLAP will open a new view of your machine with any nondeterministic states highlighted. If nothing is highlighted, your machine passes this check. If any state is highlighted, inspect its outgoing transitions and remove or fix the conflicting one(s).

**Method 2: Watch the number of simultaneous configurations while stepping.** A deterministic machine only ever has *one* active configuration at any point during simulation. When you step through your machine (**Input → Step**), if you ever see more than one configuration displayed at the same time – or if JFLAP asks you to freeze/remove configurations because there are too many active at once to display – that is a clear sign your machine has taken a nondeterministic branch somewhere. A correctly built deterministic machine for this assignment should never show more than one configuration box during stepping.

Fix any state flagged by either method before submitting. The autograder performs an equivalent check on the raw transition table of your `.jff` file, so a machine that fails determinism in JFLAP will also fail it (and receive 0 points for that problem) when graded.

## Decision Problems (Problems 1–4)

For these problems, your machine does not need to leave any particular output on the tape – it only needs to halt in an accepting state on strings that belong to the language, and halt in a non-accepting state (or fail to halt) on strings that do not. All four use the input alphabet $\{0, 1\}$.

1. **Contains a 1.** (4 points) Accept if and only if the input string contains at least one `1` somewhere.
   *Hint:* Scan right from the start. On each step: if you read `1`, transition straight to an accepting state – you're done, no need to keep scanning. If you read `0`, stay in the same scanning state and move right. If you read blank before ever seeing a `1`, you'll simply have no transition to take and the machine halts non-accepting. Two states total, plus your accept state. No tape writes are needed at all or, more precisely, it doesn't really matter what you write.
   Files: `problem1_contains1.jff`, `problem1_contains1_tests.txt`

2. **Ends with 0.** (4 points) Accept if and only if the last symbol of the input is `0`. The empty string does not end in `0` and should be rejected.
   *Hint:* First scan right until you hit the blank symbol – this is the standard "find the end of the tape" idiom you'll reuse frequently: stay in a scanning state on any input symbol, move right, and leave that state only when you read blank. Once you're at the blank, move the head left one cell and check what's there: `0` means accept, `1` means reject.
   Files: `problem2_endswith0.jff`, `problem2_endswith0_tests.txt`

3. **Even number of 1s.** (4 points) Accept if and only if the input contains an even number of `1`s (zero counts as even, so the empty string and any all-`0` string should be accepted).
   *Hint:* Use two states as a parity counter: one meaning "an even number of 1s seen so far" and one meaning "odd." On `0`, stay in the same state and move right. On `1`, switch states

and move right. When you reach blank, accept if you're in the "even" state. No tape writes are needed – the states themselves are your memory.
Files: `problem3_evenones.jff`, `problem3_evenones_tests.txt`

4. **Equal number of 0s and 1s.** (4 points) Accept if and only if the input has the same number of `0`s as `1`s.
*Hint:* This is the one problem in this set that genuinely needs you to write symbols outside $\{0, 1\}$, because you need a way to remember which symbols you've already counted across repeated passes over the tape. Pick two markers, e.g. `X` for "a 0 that's been counted" and `Y` for "a 1 that's been counted." or perhaps just one symbol... your call. Loop: scan right for the leftmost unmarked `0`; if you find one, mark it `X`, return the head to the start, then scan right for the leftmost unmarked `1` and mark it `Y` the same way, then return to the start again. Repeat. If you ever go looking for a `0` and find none left, but an unmarked `1` still remains (or vice versa), reject – the counts don't match. If neither remains, accept. This will take many passes over the tape for longer strings, which is fine. Don't worry about efficiency... simple and correct gets full credit and debugging a malfunctioning complicated TM is pretty miserable to be honest.
Files: `problem4_equalcount.jff`, `problem4_equalcount_tests.txt`

## Transducer Problems (Problems 5–6)

For these problems your machine computes a value rather than just accepting or rejecting. Both use the input alphabet $\{0, 1\}$.

### How output is read

A transducer's output is **not** read starting from the left edge of the tape. Instead: **when your machine halts, the grader reads the output starting at whatever cell the head is on at that moment, and continues rightward until it hits the first blank symbol.** This means the position of your head when the machine halts matters just as much as what you wrote to the tape. If your machine finishes a left-to-right pass and halts with the head sitting past the end of its output, the grader will read an empty (or wrong) result even if the correct answer is sitting right there on the tape a few cells to the left.

**Because of this, both transducers below must return the head to the leftmost cell of their output before halting.** A common pattern: do your left-to-right work, and when you hit the blank on the right side, switch to a "return" state that moves the head left (rewriting each symbol unchanged) until it reads a blank – meaning it's gone one cell past the start of the input – then take one more step right and halt there.

Neither transducer problem requires a final/accepting state. Since these machines aren't deciding accept/reject, they can simply halt by running out of applicable transitions in whatever state they end in – the grader only checks the tape output, not which state you stopped in.

### A warning about JFLAP's Multiple Run

You can load `problem5_flipbits_tests.txt` or `problem6_deletedup_tests.txt` into **Input → Multiple Run (Transducer)** just like the decision problems, and JFLAP will dutifully show you an Accept/Reject column. **Ignore that column for these two problems.** Accept/Reject there only reflects whether your machine happened to halt in a state you marked final – and since

transducers don't need a final state at all, **"accepting" every input (or every input showing the same result) is not enough and proves nothing about whether your output is correct.** A machine that does absolutely nothing to the tape can still show up as accepting or rejecting consistently while producing entirely wrong output. The only thing that is actually graded is the tape contents at halt, which Multiple Run does not show you at all *but Multiple Run (Transducer) does.*

5. **Flip all bits.** (4 points) Given a string over $\{0, 1\}$, produce its bitwise complement: every `0` becomes `1` and every `1` becomes `0`.
   *Hint:* Single left-to-right pass: read a symbol, write its opposite, move right, repeat until blank. Then switch to a return state that moves left (leaving symbols unchanged) until it reads blank again, then step right once and halt – landing the head back on the first symbol of your flipped output, per the output convention above.
   Files: `problem5_flipbits.jff`, `problem5_flipbits_tests.txt`, `problem5_flipbits_expected_outputs.t`

6. **Delete duplicate runs.** (5 points) Given a string over $\{0, 1\}$, collapse every maximal run of identical consecutive symbols down to a single symbol. For example, `000111000` becomes `010`, and `0011` becomes `01`.
   *Hint:* This problem is nontrivial at this point; my solution had 10 states. You only need to remember the last symbol you *kept* – your state can track this. Scan right: for each new symbol, compare it to the last symbol you kept. If it matches, delete it (overwrite that cell with blank) and move on. If it doesn't match, keep it, remember it as your new "last kept symbol," and move on. When you reach the first blank, you're done scanning – but the tape now likely has *gaps* where deleted symbols used to be, and those gaps break the "read to the first blank" output convention. **You must close those gaps** by shifting everything left to squeeze out the blanks before you halt, then return your head to the leftmost cell of the now-compacted output as described above. An alternate strategy might write a marker symbol (often '#' or '$') at the end of the string, which will be the symbol you, eventually, accept/halt on after working on the original string to the left the marker symbol you wrote and writing your solution to the right of marker symbol.
   Files:
   `problem6_deletedup.jff`,
   `problem6_deletedup_tests.txt`,
   `problem6_deletedup_expected_outputs.txt`

## Test Data Files

**Decision problems (1–4):** each `*_tests.txt` file contains exactly approximately 20 lines, more for the last problem. For the deciders lines 1–10 are strings your machine must **accept**; lines 11–20 are strings your machine must **reject**. A blank line represents the empty string ($\lambda$). You can load one of these files directly into JFLAP's batch simulator via **Input** → **Multiple Run**, then **File** → **Open** the `*_tests.txt` file from within that window, and click **Run Inputs** to see accept/reject results for all 20 strings at once.

**Transducer problems (5–6):** each transducer problem gives you *two* files instead of one:

- `*_tests.txt` – one input string per line, and nothing else. This is the file to load into JFLAP: use **Input** → **Multiple Run (Transducer)**, **File** → **Open** this file, and **Run Inputs** to

quickly check that your machine halts cleanly on every sample input (no crashes, no getting stuck). Remember the warning above – the Accept/Reject column it shows you is not what's being graded.

- `*_expected_outputs.txt` – the exact output your machine must leave on the tape for each input, one per line, in the *same order* as `*_tests.txt` (so line $k$ of one file corresponds to line $k$ of the other). This file is not meant to be loaded into JFLAP – it's for you to read directly in a text editor. For each input you test, look up the matching line here and use **Input** → **Step** or **Input** → **Fast Run** in JFLAP to check by eye that your tape actually ends up showing that string (read from the head's position at halt, per the convention above). The command-line grading script described below performs this same comparison automatically for every line.

**Do not edit any `*_tests.txt` or `*_expected_outputs.txt` file.** They will be replaced with the original versions before grading, so any changes you make will simply be discarded – editing them will only make it harder for you to debug locally with an inaccurate picture of what will actually be graded.

## Testing Your Machines

You have two complementary ways to test your work as you go:

1. **Inside JFLAP (interactive).** Use **Input** → **Step** to walk through individual strings one transition at a time and watch the tape, **Input** → **Fast Run** for a quick single-string check, or (for decision problems only) **Input** → **Multiple Run** loaded with the provided `*_tests.txt` file to check all 20 strings at once.

2. **From the command line (automated).** This assignment folder includes `test_tm.py`, the same grading script that will be used to grade your submission. From inside this folder, run:

   ```
   python3 test_tm.py
   ```

   This parses all six of your `.jff` files, checks each for determinism, tests each against its test file (accept/reject for Problems 1–4, exact tape output for Problems 5–6), and prints a full report along with your total score out of 25. Run this as often as you like while working – it is exactly what will be run when your assignment is graded (with the test files and this script itself refreshed to their original versions first, so make sure you are testing against the originals).

I recommend working in JFLAP while building. Remember that you must have an initial state before testing.

## Grading

Point values are listed with each problem above (4 points each for Problems 1–5, 5 points for Problem 6, totaling 25 points). For a given problem, you receive full credit only if *both* of the following hold:

- Your submitted machine is deterministic (see above), **and**

- Your machine produces the correct result on every test case for that problem (correct accept/reject for Problems 1–4, or exact matching tape output for Problems 5–6).

Otherwise, that problem receives 0 points. There is no partial credit within a single problem. These problems are basically fundamental and I'd say all but the last are trivial other than getting used to JFLAP.

## Submission Instructions

1. Open **YOURUSERNAME.txt** and put your university username as the *first line* of the file (and only your username – nothing else on that line). This is how the grading script identifies your submission. Forget to do this and everything is correct then you will probably get a -30% for lateness.

2. Build and save your six Turing machines into the provided `.jff` files (`problem1_contains1.jff` through `problem6_deletedup.jff`), overwriting the empty templates. Just open the file with JFLAP and save with the same name.

3. Verify all six machines are deterministic and pass their test data using `python3 test_tm.py` as described above.

4. Submit this entire folder as a single zip file. Do not rename any of the provided files as this will break the python script.

## Citation

Original instructions and testing script created by Claude.ai, with additional hints and clarifications added by James O'Reilly. Test script and files required some iteration. All problems done independently by James O'Reilly to test the script and improve instructions.